Experiment 7(b) - OpenMP Task Management

Parallel Programming Using OpenMP - Step-by-Step Practice Guide

Aim: To implement parallel programming using OpenMP.

Record workflow: install the OpenMP library in Ubuntu and run six sample programs: Hello World, loop-level parallelism with reduction, private, firstprivate, lastprivate and critical section.

Step 1 - Install the OpenMP library

```
sudo apt-get install libomp-dev
```

The record compiles its C++ examples using **g++ -fopenmp**. Verify the compiler:

```
g++ --version
```

Sample 1 - Hello World using multiple threads

```
nano hello_omp.cpp
```

```
#include <omp.h>
#include <iostream>

int main() {
    omp_set_num_threads(4);
    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        int total_threads = omp_get_num_threads();
        std::cout << "Hello World from thread " << thread_id
                  << " out of " << total_threads << " threads.\n";
    }
    return 0;
}
```

```
g++ -fopenmp hello_omp.cpp -o hello_omp.out
./hello_omp.out
```

You should get four Hello World lines. Thread order can differ because the threads run in parallel.

Sample 2 - Loop-level parallelism with reduction

The source screenshot in your record is cut off after the beginning of the timing section. The following is a **practice completion** reconstructed from the visible code, the shown output, and the same reduction logic, so you can compile and practise it.

```
nano loop_parallel.cpp
```

```
#include <omp.h>
#include <iostream>
#include <chrono>
#define THREAD_COUNT 5
using namespace std;

int serial_sum(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) sum += i;
    return sum;
}

int parallel_sum(int n) {
    int sum = 0;
    #pragma omp parallel for reduction(+:sum)
    for (int i = 1; i <= n; i++) sum += i;
    return sum;
}

int main() {
    omp_set_num_threads(THREAD_COUNT);
    int n = 100000000;

    auto start_time1 = chrono::high_resolution_clock::now();
    int ser_sum = serial_sum(n);
    auto end_time1 = chrono::high_resolution_clock::now();
    chrono::duration<double> duration1 = end_time1 - start_time1;

    auto start_time2 = chrono::high_resolution_clock::now();
    int par_sum = parallel_sum(n);
    auto end_time2 = chrono::high_resolution_clock::now();
    chrono::duration<double> duration2 = end_time2 - start_time2;

    cout << "Serial Result: " << ser_sum << endl;
    cout << "Time taken for serial execution: " << duration1.count() << endl;
    cout << "Parallel Result: " << par_sum << endl;
    cout << "Time Taken for Parallel Execution: " << duration2.count() << endl;
    return 0;
}
```

```
g++ -fopenmp loop_parallel.cpp -o loop_parallel.out
./loop_parallel.out
```

The record shows the same serial and parallel result followed by separate execution times. Exact timing values will differ on every computer.

The result 987459712 occurs because the record uses a 32-bit **int** for a very large sum; this guide preserves that behaviour for practice.

Sample 3 - Using private

```
nano private.cpp
```

```
#include <omp.h>
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    #pragma omp parallel private(x)
    {
        cout << "Thread " << omp_get_thread_num()
             << " at starting has x = " << x << endl;
        x = omp_get_thread_num();
        cout << "Thread " << omp_get_thread_num()
             << " after initialization has x = " << x << endl;
    }
    cout << "Outside parallel region, x = " << x << endl;
    return 0;
}
```

```
g++ -fopenmp private.cpp -o private.out
./private.out
```

private(x) gives every thread its own independent copy of x. The outer x remains 10.

Sample 4 - Using firstprivate

```
nano first_private.cpp
```

```
#include <omp.h>
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    #pragma omp parallel firstprivate(x)
    {
        cout << "Thread " << omp_get_thread_num()
             << " starts with x = " << x << endl;
        x += omp_get_thread_num();
        cout << "Thread " << omp_get_thread_num()
             << " ends with x = " << x << endl;
    }
    cout << "Outside parallel region, x = " << x << endl;
    return 0;
}
```

```
g++ -fopenmp first_private.cpp -o first_private.out
./first_private.out
```

firstprivate(x) gives each thread a private copy initialized from the original value 10.

Sample 5 - Using lastprivate

```
nano last_private.cpp
```

```
#include <omp.h>
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    #pragma omp parallel for lastprivate(x)
    for (int i = 0; i < 4; i++) {
        x = i;
        cout << "Thread " << omp_get_thread_num()
             << " working on iteration " << i
             << " sets x = " << x << endl;
    }
    cout << "Outside parallel region, x = " << x << endl;
    return 0;
}
```

```
g++ -fopenmp last_private.cpp -o last_private.out
./last_private.out
```

The record shows the final outer value as **x = 3**, copied from the logically last loop iteration.

Sample 6 - Using critical section

```
nano critical.cpp
```

```
#include <omp.h>
#include <iostream>
using namespace std;

int main() {
    int total_sum = 0;
    #pragma omp parallel for
    for (int i = 0; i < 1000; i++) {
        int local_sum = i;
        #pragma omp critical
        {
            total_sum += local_sum;
        }
    }
    cout << "Total sum: " << total_sum << endl;
    return 0;
}
```

```
g++ -fopenmp critical.cpp -o critical.out
./critical.out
```

Expected result:

```
Total sum: 499500
```

Key OpenMP concepts to remember

• **#pragma omp parallel** creates a team of threads.

• **omp_get_thread_num()** returns the current thread ID.

• **reduction(+:sum)** safely combines partial sums.

• **private(x)** gives each thread a private copy.

• **firstprivate(x)** initializes each private copy from the original value.

• **lastprivate(x)** copies the logically last loop value back to the outer variable.

• **#pragma omp critical** lets only one thread at a time execute the protected block.

Quick exam rehearsal

```
sudo apt-get install libomp-dev

g++ -fopenmp hello_omp.cpp -o hello_omp.out
./hello_omp.out

g++ -fopenmp loop_parallel.cpp -o loop_parallel.out
./loop_parallel.out

g++ -fopenmp private.cpp -o private.out
./private.out

g++ -fopenmp first_private.cpp -o first_private.out
./first_private.out

g++ -fopenmp last_private.cpp -o last_private.out
./last_private.out

g++ -fopenmp critical.cpp -o critical.out
./critical.out
```

Common mistakes

• Compiling without **-fopenmp**.

• Forgetting **#include <omp.h>**.

• Expecting thread output to always appear in the same order.

• Confusing **private**, **firstprivate** and **lastprivate**.

• Updating shared data without a reduction or critical section when synchronization is required.

Result

Thus parallel processing using OpenMP was completed successfully.